ЛАБОРАТОРИ АЖИЛ №3

Оноо: 3

---

1. Зорилго

Энэхүү лабораторийн ажлын зорилго нь дундын өгөгдөлтэй (shared data) ажиллахад үүсэх асуудлуудыг OpenMP-ийн үндсэн механизмууд дээр тулгуурлан судалж, тэдгээрийн гүйцэтгэлийн ялгааг практикт хэмжих явдал юм.

Ажлыг гүйцэтгэсний дараа оюутан дараах чадвартай болно:

- Race condition үүсэх мөн чанарыг кодын түвшинд тайлбарлах
- Синхрончлолын (critical) зардлыг тоогоор хэмжих
- reduction-ийн давуу талыг critical-тай харьцуулж батлах
- Speedup (хурдны өсөлт) ба Efficiency (үр ашиг)-ийг тооцоолж, график хэлбэрээр дүгнэх
- First touch-ийн нөлөөг өөрийн систем дээр шалгаж, үр дүнг тайлбарлах

2. Шаардлагатай орчин ба компайл хийх

macOS (Apple Silicon ба Intel)

macOS-ийн clang нь OpenMP-г дотроо агуулдаггүй тул libomp-ийг эхлээд суулгаарай:

```
brew install libomp

clang -Xpreprocessor -fopenmp \
      -I$(brew --prefix libomp)/include \
      -L$(brew --prefix libomp)/lib -lomp \
      -O3 -o lab3 lab3.c timer.c
```

Linux

```
gcc -fopenmp -O3 -o lab3 lab3.c timer.c
```

Ажиллуулах

```
export OMP_NUM_THREADS=4
./lab3
```

Анхаар: -O3 оновчлолын флагийг заавал ашиглана. -O0 дээр хэмжилт тохирохгүй. Хугацааг тооцоолохдоо сурах бичгийн 17.2 хэсэгт өгөгдсөн timer.c файлыг ашиглана.

3. Даалгавар

80,000,000 (80 сая) элементтэй **`a, b, c`** гэсэн **`double`** төрлийн массив зарлана. **a** массивын бүх элементэд 1.0, **b** массивын бүх элементэд 2.0 гэсэн утга оноож өгнө.

Дараа нь хоёр үйлдлийг доорх хувилбаруудаар тус тус гүйцэтгэж, хугацааг хэмжинэ:

1. Вектор нэмэх: c[i] = a[i] + b[i]
2. Нийлбэр олох: sum = c[0] + c[1] + ... + c[N-1]

Зөв хариу: `c[i] = 3.0` тул нийлбэр нь **240,000,000.0** байх ёстой.

> Санах ойн шаардлага: 80,000,000 × 8 байт × 3 массив ≈ 1.9 GB. Хэрэв таны машин удаашрах эсвэл санах ой хүрэхгүй бол N-ийг 20,000,000 болгон бууруулж болно. Энэ тохиолдолд тайландаа ашигласан N-ийн утгаа заавал бичнэ.

4. Хэрэгжүүлэх хувилбарууд

Хэсэг А: Вектор нэмэх (3 хувилбар)

A1 — Tseq (дараалсан)

```
for (int i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```

A2 — Tprag (параллель, инициализаци цуваа)

```
// Инициализаци — ЦУВАА (pragma байхгүй)
for (int i = 0; i < N; i++) {
    a[i] = 1.0;  b[i] = 2.0;
}

#pragma omp parallel for
for (int i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```

A3 — Tft (параллель, инициализаци мөн параллель — First Touch)

```
// Инициализаци — МӨН ПАРАЛЛЕЛЬ
#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++) {
    a[i] = 1.0;  b[i] = 2.0;
}

#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++)
    c[i] = a[i] + b[i];
```

Хэсэг Б: Нийлбэр олох (4 хувилбар)

B1 — Tseq (дараалсан)

```
double sum = 0.0;
for (int i = 0; i < N; i++)
    sum += c[i];
```

B2 — Trace (хамгаалалтгүй — Race Condition)

```
double sum = 0.0;
#pragma omp parallel for
for (int i = 0; i < N; i++)
    sum += c[i];           // ХАМГААЛААГҮЙ
```

B3 — Tcrit (critical ашигласан)

```
double sum = 0.0;
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    #pragma omp critical
    sum += c[i];
}
```

B4 — Tred (reduction ашигласан)

```
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < N; i++)
    sum += c[i];
```

5. Хэмжилт

1. Эхлээд дараалсан хувилбаруудыг (A1, B1) хэмжинэ. Эдгээрт OpenMP байхгүй тул урсгалын тооноос хамаарахгүй — түс бүрийг НЭГ л утгаар бүртгэнэ (Хүснэгт 1).
2. Дараа нь параллель хувилбаруудыг OMP_NUM_THREADS = 2, 4, 8 болгон өөрчилж хэмжинэ (Хүснэгт 2, 3).
3. Хувилбар бүрийг 3 удаа ажиллуулж, дундаж хугацааг хүснэгтэд бичнэ.
4. Хэмжилтийн үед бусад том хэмжээтэй программыг хаана уу. Төхөөрөмжээ цэнэгтэй, тогтвортой горимд байлгана.
5. Хугацааг зөвхөн тооцооллын хэсэгт хэмжинэ — инициализаци, санах ой хуваарилалтыг оруулахгүй (A3 хувилбарт инициализацийг тусад нь хэмжиж болно).

6. Үр дүнгийн хүснэгтүүд

Хүснэгт 1. Суурь хэмжилт — дараалсан хувилбарууд (Tseq)

A1 ба B1 хувилбарт OpenMP огт байхгүй тул урсгалын тооноос хамаарахгүй. Түс бүрийг 3 удаа ажиллуулж дундажлаад, НЭГ утга гаргана. Энэ утгыг доорх бүх Speedup-ийн тооцоонд ижил хуваарьт ашиглана.

Дараалсан хувилбар	Хугацаа (сек)
A1 — Вектор нэмэх: c[i] = a[i] + b[i]	
B1 — Нийлбэр олох: sum += c[i]	

Хүснэгт 2. Вектор нэмэх: параллель хувилбарууд

Урсгалын тоо	Tprag (сек)	Tft (сек)
2		
4		
8		

> Хэрэв Tprag ба Tft хоёрын хооронд ялгаа бараг гарахгүй бол энэ нь ХЭВИЙН. First touch нь зөвхөн олон сокеттэй (NUMA) системд нөлөөтэй. Өөрийн системийг шалгаад (Linux: numactl --hardware, macOS: sysctl hw.memsize), яагаад ялгаа гараагүйг дүн шинжилгээнд тайлбарлана.

Хүснэгт 3. Нийлбэр олох: параллель хувилбарууд

Урсгалын тоо	Trace (сек)	Tcrit (сек)	Tred (сек)
2			
4			
8			

> Урсгалын тоо 1 байх мөр нь Tseq биш гэдгийг анхаар: энэ нь OpenMP-тэй кодыг нэг урсгалаар ажиллуулсан хугацаа. Tseq-ээс арай удаан гарвал энэ нь OpenMP-ийн нэмэлт зардал (overhead) болно — үүнийг тайландаа тэмдэглэ.

Хүснэгт 4. Race condition-ийн үр дүн (3 удаа ажиллуулсан)

Хувилбар B2-ийг урсгалын тоо бүр дээр 3 удаа ажиллуулж, гарсан нийлбэрийг бичнэ.

Урсгалын тоо	Ажиллуулалт 1	Ажиллуулалт 2	Ажиллуулалт 3	Зөв хариу
2				240000000
4				240000000
8				240000000

Хүснэгт 5. Speedup ба Efficiency

Урсгалын тоо	S(crit)	E(crit)	S(red)	E(red)
2				
4				
8				

7. Тооцооллын томьёо

```
Speedup     :   S(P) = Tseq / TP
Efficiency  :   E(P) = S(P) / P

   P    – урсгалын тоо
   Tseq – дараалсан хувилбарын хугацаа
   TP   – P урсгалтай үеийн хугацаа
```

Efficiency-г хувиар (%) илэрхийлж бичиж болно. E = 1.0 (100%) бол төгс scaling гэсэн үг.

8. График

Excel, Python (matplotlib) эсвэл өөр тохиромжтой хэрэгсэл ашиглан дараах 2 графикийг зурна:

1. График 1 — Speedup: X тэнхлэг нь урсгалын тоо (1, 2, 4, 8), Y тэнхлэг нь Speedup. Tcrit ба Tred хувилбаруудын муруйг нэг график дээр зэрэгцүүлнэ. Төгс scaling-ийн шугамыг (S = P) тасархай шугамаар нэмбэл илүү ойлгомжтой.

2. График 2 — Efficiency: X тэнхлэг нь урсгалын тоо, Y тэнхлэг нь Efficiency (%). Мөн хоёр хувилбарыг зэрэгцүүлнэ.

9. Асуулт

Доорх асуултуудад өөрийн хэмжсэн бодит тоонд тулгуурлан хариулна. Зөвхөн онол давтах хариултыг тооцохгүй.

1. Урсгалын тоо нэмэгдэхэд Tcrit яагаад сайжрахгүй, эсвэл бүр мууддаг вэ? Механизмыг тайлбарла.
2. Tred яагаад Tcrit-ээс эрс хурдан вэ? Хоёр механизмын ялгааг кодын түвшинд тайлбарла.
3. Хүснэгт 4-д гарсан гурван үр дүн хоорондоо ялгаатай байна уу? Хэрэв тийм бол энэ нь юуг харуулж байна вэ?
4. Урсгалын тоо нэмэгдэхэд Efficiency хэрхэн өөрчлөгдөж байна? Яагаад ийм хандлага ажиглагдаж байгааг Amdahl-ийн хуультай холбон тайлбарла.
5. Tprag ба Tft хоёрын хооронд ялгаа гарсан уу? Таны системд хэдэн NUMA зангилаа байна вэ? Үр дүнг системийн бүтэцтэй холбон тайлбарла.
6. Нэмэлт (сонголтоор): B2 хувилбарыг -O0 болон -O3 флагаар тус тус компайл хийж ажиллуулаад, гарсан буруу үр дүнгүүдийг харьцуул. Яагаад өөр байна вэ?

10. Даалгаврыг илгээх

- Эх код (lab3.c, timer.c, timer.h)
- Тайлан (PDF эсвэл Word) — бүх хүснэгт бөглөсөн байх
- 2 график (тайлан дотор эсвэл тусдаа файл)
- Дүн шинжилгээний асуултуудын хариулт
- Тайлангийн эхэнд: ашигласан машины үзүүлэлт (CPU загвар, цөмийн тоо, RAM), компайлерын хувилбар, ашигласан N-ийн утга

11. Үнэлгээний шалгуур

Шалгуур	Эзлэх хувь
Бүх хувилбар зөв хэрэгжсэн, компайл алдаагүй	25%
Хэмжилтийг журмын дагуу гүйцэтгэж, хүснэгтийг бүрэн бөглөсөн	25%
Speedup, Efficiency-г зөв тооцоолж, 2 график зурсан	20%
Дүн шинжилгээний асуултуудад бодит тоогоор баталгаажуулж хариулсан	25%
Код цэвэр, тайлбартай; тайлан эмх цэгцтэй	5%

Анхаарах зүйлс

- malloc-ийн буцаасан утгыг NULL эсэхийг заавал шалгана — 1.9 GB хуваарилахад амжилтгүй болж болзошгүй.
- Хугацааны хэмжилтийг нэг удаа биш, 3 удаа хийж дундажлана. Эхний ажиллуулалт нь ихэвчлэн удаан байдаг.
- B3 хувилбар (critical) 8 урсгалтай үед МАШ удаан ажиллана — энэ нь хэвийн, яг энэ зардлыг хэмжих нь даалгаврын зорилго.
- B2 хувилбарын үр дүн ажиллуулах бүрд өөр гарна. Энэ бол алдаа биш — race condition-ийн шинж чанар.
- double төрлийн нийлбэрт бага зэргийн тоймлолтын зөрүү (жишээ: 239999999.9999) гарч болно. Энэ нь race condition биш.
- Хэрэв reduction хувилбар дараалсанаас удаан гарвал — N хэт бага эсвэл машин ачаалалтай байж магадгүй.